Agentic Sales Orchestrator - Opportunity Lookup Fix

Customer-ready relationship fix runbook with true line-by-line script explanation

Agentic Sales Orchestrator - Pending Commercial Action Opportunity Lookup Fix Runbook

Customer-ready implementation guide for the relationship fix after the initial language-code error

Item	Value
Environment	Phoenicarix-CI
Dataverse URL	https://phoenicarix-ci.crm4.dynamics.com
Solution	ASO.Core
Solution unique name	ASOCore
Target table	Pending Commercial Action
Target table logical name	aso_pendingcommercialaction
Missing component fixed	Opportunity lookup / relationship
Lookup schema name	aso_opportunityid
Relationship schema name	aso_opportunity_aso_pendingcommercialaction
Fix	Use language code 1031 instead of 1033 for relationship labels
Version	v1.0
Date	2026-05-24

Executive summary

The first Pending Commercial Action script created the table and its columns, but the Opportunity lookup relationship failed with this message:

```
FAILED LOOKUP: aso_opportunityid
The language code 1033 is not a valid language for this organization
```

We fixed the issue with a second focused script. The fix script did not recreate the table and did not recreate the columns. It only checked whether aso_opportunityid existed, then created the missing many-to-one relationship from Pending Commercial Action to Opportunity using LanguageCode = 1031.

Business objective

The relationship is required because every pending commercial action should be traceable back to the Opportunity that triggered it. Without that lookup, the table could store commercial actions, but users and automations would not have a clean parent Opportunity reference.

Technical objective

Create a Dataverse lookup column aso_opportunityid on aso_pendingcommercialaction and a many-to-one relationship named aso_opportunity_aso_pendingcommercialaction, where Opportunity is the referenced/parent table and Pending Commercial Action is the referencing/child table.

Scope

In scope:

- Read existing columns on `aso_pendingcommercialaction`.

- Skip the fix if `aso_opportunityid` already exists.
- Create relationship metadata using `CreateOneToManyRequest`.
- Use German language code `1031` for labels.
- Publish `aso_pendingcommercialaction` and `opportunity`.
- Validate the relationship in Power Apps.

Out of scope:

- Recreating the table.
- Recreating the fields.
- Changing the previous Action Type or Status choices.
- Building forms, flows, approvals, or SAP integration.

Root cause and resolution

Area	Explanation
What failed	The lookup relationship creation failed in the initial script.
Error text	The language code 1033 is not a valid language for this organization
Why it failed	The organization rejected the English label language code for this metadata operation.
What we changed	The fix script uses const int LanguageCode = 1031;
Why this worked	1031 is German, which is supported in the current environment/user interface context.
What was created	Lookup aso_opportunityid and relationship aso_opportunity_aso_pendingcommercialaction.

Architecture/context

```
Opportunity (opportunity)
  1 -> many
Pending Commercial Action (aso_pendingcommercialaction)
  lookup column: aso_opportunityid
```

From the user perspective, this means a Pending Commercial Action can now be related to a specific Opportunity. From a technical perspective, automations can use the lookup to query all pending commercial actions for one opportunity or write back the opportunity context when staging an action.

Step-by-step implementation summary

1. The initial script created table and columns.

2. The lookup failed because of language code 1033.

3. We left the successful table/column changes in place.

4. We replaced Program.cs with a focused fix script.

5. The fix script used LanguageCode = 1031.

6. The fix script checked whether aso_opportunityid already existed.

7. The fix script created the one-to-many relationship and lookup.

8. The script published both Pending Commercial Action and Opportunity metadata.

9. We validated the relationship in Power Apps under the Relationships area.

Validation checklist

Check	Expected result
Terminal output	CREATED LOOKUP: aso_opportunityid
Relationship visible	aso_opportunity_aso_pendingcommercialaction appears
Relationship display	Opportunity
Related table	Opportunity / Verkaufschance
Type	Many-to-one from Pending Commercial Action to Opportunity
Publish completed	Script ends with Done. Validate...

Troubleshooting guide

Symptom	Likely cause	Action
1033 is not valid	English not enabled/supported for this metadata label operation	Use LanguageCode = 1031
Lookup already exists	Fix script was rerun	Safe; script skips creation
Relationship not visible	Browser cache or unpublished metadata	Refresh maker portal; publish customizations again
Build error	Program.cs incomplete	Re-paste the complete fix script
Permission error	User cannot customize metadata	Use System Administrator/System Customizer account

Rollback/recovery approach

If the lookup was created incorrectly, remove the relationship only if no data or dependencies depend on it. If rows already reference opportunities, export data or remove references before deleting. In production, relationship changes should be controlled by managed solution versioning and environment restore planning.

What each audience should understand

Non-technical person

We connected the waiting-room record to the sales opportunity it belongs to. This makes it possible to understand which deal the pending commercial action belongs to.

Product Owner

The fix completed traceability. Pending commercial actions are now business-linked to opportunities, which supports approval, audit, and future SAP-safe automation.

Developer / technical consultant

The fix used CreateOneToManyRequest with OneToManyRelationshipMetadata and LookupAttributeMetadata. The important technical change was using LanguageCode = 1031 to create the required relationship labels in this organization.

Part 2 - Full fix script documentation with line-by-line explanation

Line	Code	Explanation in simple language	Technical explanation	Why it matters	Common mistake / warning
1	using Microsoft.Crm.Sdk.Messages;	Imports the Microsoft.Crm.Sdk.Messages namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
2	using Microsoft.PowerPlatform.Dataverse.Client;	Imports the Microsoft.PowerPlatform.Dataverse.Client namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
3	using Microsoft.Xrm.Sdk;	Imports the Microsoft.Xrm.Sdk namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.

4	using Microsoft.Xrm.Sdk.Messages;	Imports the Microsoft.Xrm.Sdk.Messages namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
5	using Microsoft.Xrm.Sdk.Metadata;	Imports the Microsoft.Xrm.Sdk.Metadata namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
6	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
7	const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";	Defines the target Dataverse environment URL.	ServiceClient uses this URL to connect to Phoenicarix-CI.	Prevents accidental schema creation in the wrong tenant/environment.	Wrong URL creates components in the wrong environment or fails authentication.
8	const string SolutionUniqueName = "ASOCore";	Defines the technical solution unique name: ASOCore.	CreateEntityRequest, CreateAttributeRequest, and CreateOneToManyRequest use this to associate metadata with ASO.Core.	Keeps created components inside the project solution for ALM/export.	Using display name instead of unique name can leave components outside the intended solution.
9	const string EntityLogicalName = "aso_pendingcommercialaction";	Defines the target table logical name.	The script uses aso_pendingcommercialaction as the Dataverse table logical name.	Every table, column, relationship, and publish request depends on this value.	Wrong logical name targets the wrong table or causes metadata request failures.
10	const int LanguageCode = 1031;	Sets German language code 1031 for labels.	Label(...) uses this language code for relationship and lookup labels.	This resolved the missing Opportunity lookup relationship creation in the German-language organization.	Using unsupported language codes can cause metadata creation failures.
11	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
12	var connectionString =	Starts the Dataverse connection string definition.	The following lines define OAuth authentication, URL, login behavior, client ID, and redirect URI.	The script needs an authenticated ServiceClient to execute metadata operations.	Incomplete connection strings prevent login or service initialization.
13	$@"AuthType=OAuth;	Specifies OAuth authentication.	OAuth is the interactive sign-in method for the local Dataverse SDK connection.	Allows a maker/admin to sign in with their Power Platform account.	Wrong auth type may require unavailable secrets or fail on Mac/local use.
14	Url={DataverseUrl};	Injects the configured Dataverse URL into the connection string.	Uses the DataverseUrl constant rather than hardcoding the URL multiple times.	Makes environment targeting explicit and maintainable.	If the URL is wrong, all metadata operations target the wrong environment.
15	LoginPrompt=Auto;	Allows interactive login when needed.	ServiceClient can open a login flow if no valid token is cached.	Practical for a Mac-based maker/developer working locally.	Without a login prompt, the script may fail silently when no token is available.
16	ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;	Sets the Microsoft public client ID used by Dataverse tooling scenarios.	ServiceClient uses it for the interactive OAuth flow.	Avoids creating a custom app registration for this MVP metadata script.	Changing it to an invalid client ID breaks authentication.
17	RedirectUri=http://localhost";	Defines the local redirect URI for interactive OAuth.	After sign-in, authentication returns to localhost so the SDK can continue.	Required by the OAuth flow used in this local script.	Incorrect redirect URI can block authentication completion.
18	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
19	using var service = new ServiceClient(connectionString);	Imports the var service = new ServiceClient(connectionString) namespace.	Allows the script to reference Dataverse SDK classes without fully qualified names.	Required for SDK request classes, metadata types, labels, and the ServiceClient connection.	Missing namespace causes compiler errors for the related SDK types.
20	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
21	if (!service.IsReady)	Checks whether connection initialization succeeded.	Prevents metadata operations when authentication or connectivity failed.	Protects the environment from partial/undefined behavior.	Skipping this check can lead to confusing failures later in the script.
22	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
23	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output to red.	Used for failure messages so operators can identify errors immediately.	Improves operational readability.	If colors are not reset, later messages can remain incorrectly colored.
24	Console.WriteLine("Connection failed:");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
25	Console.WriteLine(service.LastError);	Prints the Dataverse connection error.	ServiceClient.LastError provides diagnostic detail when connection fails.	Needed for troubleshooting URL, login, permission, or token issues.	Without it, the operator sees only a generic failure.
26	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
27	return;	Stops script execution.	Used after a connection failure or after skipping lookup creation.	Prevents unsafe continuation or exits a helper method when work is unnecessary.	Removing it can cause the script to run into invalid state.
28	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
29	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
30	Console.WriteLine($"Connected to: {DataverseUrl}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
31	Console.WriteLine($"Target solution: {SolutionUniqueName}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
32	Console.WriteLine($"Target table: {EntityLogicalName}");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
33	Console.WriteLine("Fixing missing Opportunity lookup only...");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
34	Console.WriteLine();	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
35	(blank)	Blank readability line.	No runtime behavior; separates logical code	Improves maintainability and visual	None.

			blocks.	scanning.	
36	var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);	Reads existing columns on the target table.	The helper retrieves Attribute metadata and returns logical names.	Enables idempotent reruns that skip already-created columns.	Without this, reruns would fail on duplicate column names.
37	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
38	if (existingAttributes.Contains("aso_opportunityid"))	Checks whether a column already exists.	Uses the current table metadata returned earlier.	Makes the script rerunnable after partial success.	Without it, duplicate column errors stop progress.
39	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
40	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal output to yellow.	Used for skip messages when a component already exists.	Helps distinguish safe idempotent skips from failures.	Purely cosmetic, but useful during reruns.
41	Console.WriteLine("SKIP existing lookup: aso_opportunityid");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
42	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
43	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
44	else	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
45	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
46	try	Starts a protected operation block.	Exceptions inside the block are caught and logged.	Prevents one failing field from hiding the exact cause.	Without try/catch, the script may crash with less context.
47	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
48	Console.WriteLine("Creating Opportunity lookup relationship...");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
49	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
50	var relationship = new OneToManyRelationshipMetadata	Starts a one-to-many relationship definition.	From Opportunity to Pending Commercial Action: one Opportunity can have many pending actions.	This models the business relationship correctly.	Wrong relationship direction affects forms, lookups, and related records.
51	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
52	SchemaName = "aso_opportunity_aso_pendingcommercialaction",	Sets the relationship schema name.	Dataverse stores relationships with schema names.	Allows the relationship to be identified and transported in solutions.	Changing the name creates a different relationship.
53	ReferencedEntity = "opportunity",	Sets Opportunity as the parent/referenced table.	Opportunity is the table being looked up to.	Each pending action belongs to one opportunity.	Wrong referenced table breaks the business model.
54	ReferencingEntity = EntityLogicalName,	Sets Pending Commercial Action as the child/referencing table.	The lookup column lives on the Pending Commercial Action table.	This lets a pending action point to an Opportunity.	Wrong referencing entity places the lookup elsewhere.
55	AssociatedMenuConfiguration = new AssociatedMenuConfiguration	Defines how related records appear in navigation.	Controls the related menu label/group/order in model-driven apps.	Improves user navigation from Opportunity to pending actions.	Label language issues can fail this part of relationship creation.
56	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
57	Behavior = AssociatedMenuBehavior.UseLabel,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
58	Group = AssociatedMenuGroup.Details,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
59	Label = Label("Pending Commercial Actions"),	Sets the related menu label.	Uses the Label helper and current LanguageCode.	This was where unsupported 1033 contributed to the first failure.	Use an enabled organization language such as 1031 in this tenant.
60	Order = 10000	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
61	},	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
62	CascadeConfiguration = new CascadeConfiguration	Starts cascade behavior configuration.	Defines what happens to child records when the parent opportunity is assigned, deleted, merged, shared, etc.	Protects commercial audit records from accidental destructive cascades.	Aggressive cascade deletes could remove important approval/audit data.
63	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
64	Assign = CascadeType.NoCascade,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
65	Delete = CascadeType.RemoveLink,	Configures delete behavior to remove the link.	If the parent is deleted, Dataverse removes the relationship link rather than cascading deletion.	Preserves pending action records where possible.	A cascade delete could erase audit data.

66	Merge = CascadeType.NoCascade,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
67	Reparent = CascadeType.NoCascade,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
68	Share = CascadeType.NoCascade,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
69	Unshare = CascadeType.NoCascade	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
70	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
71	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
72	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
73	var lookup = new LookupAttributeMetadata	Starts lookup column metadata definition.	The lookup column stores the Opportunity reference on the child table.	Creates the visible Opportunity column in Pending Commercial Action.	Missing lookup means no direct relationship to Opportunity.
74	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
75	SchemaName = "aso_opportunityid",	Sets the lookup column schema name.	The column created on Pending Commercial Action is aso_opportunityid.	This was the missing column after the first script failed.	Wrong name breaks downstream scripts/flows expecting aso_opportunityid.
76	DisplayName = Label("Opportunity"),	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
77	Description = Label("Related Opportunity for the pending commercial action."),	Adds a metadata description.	Dataverse stores this explanation on the table or column metadata.	Helps future IT teams understand intent.	Descriptions are easy to skip but important for maintainability.
78	RequiredLevel = Optional()	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
79	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
80	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
81	var request = new CreateOneToManyRequest	Creates the relationship creation request.	Combines the relationship metadata and lookup attribute into one SDK operation.	This is how Dataverse creates lookup relationships programmatically.	Incorrect metadata can create wrong relationship direction or fail validation.
82	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
83	OneToManyRelationship = relationship,	Adds the relationship metadata to the request.	Passes schema, parent/child tables, menu configuration, and cascade behavior.	Required to create the relationship.	Without it, Dataverse does not know relationship structure.
84	Lookup = lookup,	Adds lookup column metadata to the request.	Passes schema, display label, description, and required level.	Creates the actual Opportunity lookup field on the child table.	Without it, no lookup column appears.
85	SolutionUniqueName = SolutionUniqueName	Associates the created component with ASO.Core.	The SDK passes the solution unique name to the create request.	Critical for export, ALM, and keeping changes out of accidental default-only context.	Wrong solution unique name means components may not appear in ASO.Core.
86	};	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
87	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
88	service.Execute(request);	Executes a create request.	Sends CreateAttributeRequest or CreateOneToManyRequest to Dataverse.	Actually creates a column or relationship.	Failures here indicate permissions, invalid metadata, or language/choice issues.
89	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
90	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Used for successful creation/publishing messages.	Gives visual confirmation of completed metadata actions.	Purely cosmetic, but useful during long script runs.
91	Console.WriteLine("CREATED LOOKUP: aso_opportunityid");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
92	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
93	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
94	catch (Exception ex)	Starts exception handling.	Captures SDK/Dataverse errors for the current operation.	Keeps the operator informed about exact failures.	Ignoring exceptions makes recovery difficult.
95	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
96	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output to red.	Used for failure messages so operators can identify errors immediately.	Improves operational readability.	If colors are not reset, later messages can remain incorrectly colored.
97	Console.WriteLine("FAILED LOOKUP:	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to	If messages are vague, failures become

	aso_opportunityid");			troubleshoot.	harder to interpret.
98	Console.WriteLine(ex.Message);	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
99	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
100	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
101	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
102	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
103	Console.WriteLine();	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
104	Console.WriteLine("Publishing Pending Commercial Action and Opportunity customizations...");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
105	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
106	service.Execute(new PublishXmlRequest	Creates a targeted publish request.	Publishes metadata changes for specific entities rather than everything.	Makes created columns/relationships visible and usable in the maker portal.	Without publish, changes may not appear immediately.
107	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
108	ParameterXml = "<importexportxml><entities><entity>aso_pendingcommercialaction</entity><entity>opportunity</entity></entities></importexportxml>"	Defines which tables to publish.	The XML lists aso_pendingcommercialaction and opportunity.	Both sides are relevant because a relationship touches both tables.	Wrong XML may publish too little or fail.
109	});	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
110	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
111	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Used for successful creation/publishing messages.	Gives visual confirmation of completed metadata actions.	Purely cosmetic, but useful during long script runs.
112	Console.WriteLine("Done. Validate in ASO.Core -> Tables -> Pending Commercial Action -> Columns.");	Writes a status message to the terminal.	Provides an execution log for the operator.	Makes the run auditable and easier to troubleshoot.	If messages are vague, failures become harder to interpret.
113	Console.ResetColor();	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
114	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
115	static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)	Defines a helper to retrieve existing column names.	Returns a set for fast case-insensitive lookup.	Supports safe reruns and partial recovery.	If not reliable, duplicate detection fails.
116	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
117	var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest	Creates a metadata retrieval request.	Dataverse uses it to retrieve table or attribute metadata.	Needed to check existing tables/columns.	Wrong filters may return incomplete metadata.
118	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
119	LogicalName = entityLogicalName,	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
120	EntityFilters = EntityFilters.Attributes,	Requests column/attribute metadata.	The response includes existing table attributes.	Needed for existing column checks.	Using only Entity filters would not return columns.
121	RetrieveAsIfPublished = true	Reads metadata as if all customizations are published.	Helps include recent metadata changes.	Improves reliability after script-created components.	If false, recently created unpublished metadata might not be detected.
122	});	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
123	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
124	return response.EntityMetadata.Attributes	Starts building a list of existing attribute logical names.	Reads the metadata response returned by Dataverse.	Forms the basis of idempotency checks.	If the response is null, the helper would fail.
125	.Where(a => !string.IsNullOrWhiteSpace(a.LogicalName))	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
126	.Select(a => a.LogicalName!)	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse	It contributes to the controlled creation of the Pending Commercial Action table or lookup	If changed, validate compile output and Power Apps metadata carefully.

			metadata.	fix.	
127	.ToHashSet(StringComparer.OrdinalIgnoreCase);	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
128	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
129	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
130	static Label Label(string value) => new(value, LanguageCode);	Defines a helper for localized labels.	Creates Dataverse Label objects using the configured LanguageCode.	Avoids repeating label creation logic everywhere.	Using unsupported LanguageCode caused the initial lookup failure.
131	(blank)	Blank readability line.	No runtime behavior; separates logical code blocks.	Improves maintainability and visual scanning.	None.
132	static AttributeRequiredLevelManagedProperty Optional()	Defines a helper for optional fields.	Returns AttributeRequiredLevel.None wrapped in a Dataverse managed property.	Keeps most custom columns optional for MVP flexibility.	Making fields required too early can break integrations and tests.
133	{	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
134	return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.
135	}	Structural C# syntax.	Groups code into blocks or closes object initializers/statements.	Required for valid C# compilation and scope control.	Missing braces or semicolons cause build errors.
136	EOF	Executes part of the script's metadata creation logic.	This line participates in defining, checking, creating, logging, or publishing Dataverse metadata.	It contributes to the controlled creation of the Pending Commercial Action table or lookup fix.	If changed, validate compile output and Power Apps metadata carefully.

Appendix - Full fix script

```
001: using Microsoft.Crm.Sdk.Messages;
002: using Microsoft.PowerPlatform.Dataverse.Client;
003: using Microsoft.Xrm.Sdk;
004: using Microsoft.Xrm.Sdk.Messages;
005: using Microsoft.Xrm.Sdk.Metadata;
006:
007: const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";
008: const string SolutionUniqueName = "ASOCore";
009: const string EntityLogicalName = "aso_pendingcommercialaction";
010: const int LanguageCode = 1031;
011:
012: var connectionString =
013:     $@"AuthType=OAuth;
014:        Url={DataverseUrl};
015:        LoginPrompt=Auto;
016:        ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;
017:        RedirectUri=http://localhost";
018:
019: using var service = new ServiceClient(connectionString);
020:
021: if (!service.IsReady)
022: {
023:     Console.ForegroundColor = ConsoleColor.Red;
024:     Console.WriteLine("Connection failed:");
025:     Console.WriteLine(service.LastError);
026:     Console.ResetColor();
027:     return;
028: }
029:
030: Console.WriteLine($"Connected to: {DataverseUrl}");
031: Console.WriteLine($"Target solution: {SolutionUniqueName}");
032: Console.WriteLine($"Target table: {EntityLogicalName}");
033: Console.WriteLine("Fixing missing Opportunity lookup only...");
034: Console.WriteLine();
035:
036: var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);
037:
038: if (existingAttributes.Contains("aso_opportunityid"))
039: {
040:     Console.ForegroundColor = ConsoleColor.Yellow;
041:     Console.WriteLine("SKIP existing lookup: aso_opportunityid");
042:     Console.ResetColor();
043: }
044: else
```

```
{
    try
    {
        Console.WriteLine("Creating Opportunity lookup relationship...");

        var relationship = new OneToManyRelationshipMetadata
        {
            SchemaName = "aso_opportunity_aso_pendingcommercialaction",
            ReferencedEntity = "opportunity",
            ReferencingEntity = EntityLogicalName,
            AssociatedMenuConfiguration = new AssociatedMenuConfiguration
            {
                Behavior = AssociatedMenuBehavior.UseLabel,
                Group = AssociatedMenuGroup.Details,
                Label = Label("Pending Commercial Actions"),
                Order = 10000
            },
            CascadeConfiguration = new CascadeConfiguration
            {
                Assign = CascadeType.NoCascade,
                Delete = CascadeType.RemoveLink,
                Merge = CascadeType.NoCascade,
                Reparent = CascadeType.NoCascade,
                Share = CascadeType.NoCascade,
                Unshare = CascadeType.NoCascade
            }
        };

        var lookup = new LookupAttributeMetadata
        {
            SchemaName = "aso_opportunityid",
            DisplayName = Label("Opportunity"),
            Description = Label("Related Opportunity for the pending commercial action."),
            RequiredLevel = Optional()
        };

        var request = new CreateOneToManyRequest
        {
            OneToManyRelationship = relationship,
            Lookup = lookup,
            SolutionUniqueName = SolutionUniqueName
        };

        service.Execute(request);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine("CREATED LOOKUP: aso_opportunityid");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine("FAILED LOOKUP: aso_opportunityid");
        Console.WriteLine(ex.Message);
        Console.ResetColor();
    }
}

Console.WriteLine();
Console.WriteLine("Publishing Pending Commercial Action and Opportunity customizations...");

service.Execute(new PublishXmlRequest
{
    ParameterXml = "<importexportxml><entities><entity>aso_pendingcommercialaction</entity><entity>opportunity</entity></entities></importexportxml>"
});

Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Done. Validate in ASO.Core -> Tables -> Pending Commercial Action -> Columns.");
Console.ResetColor();

static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)
{
    var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest
    {
        LogicalName = entityLogicalName,
        EntityFilters = EntityFilters.Attributes,
```

```
        RetrieveAsIfPublished = true
    });

    return response.EntityMetadata.Attributes
        .Where(a => !string.IsNullOrWhiteSpace(a.LogicalName))
        .Select(a => a.LogicalName!)
        .ToHashSet(StringComparer.OrdinalIgnoreCase);
}

static Label Label(string value) => new(value, LanguageCode);

static AttributeRequiredLevelManagedProperty Optional()
{
    return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);
}
EOF
```